

	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7	8	9
2	0	2	4	6	8	10	12	14	16	18
3	0	3	6	9	12	15	18	21	24	27
4	0	4	8	12	16	20	24	28	32	36
5	0	5	10	15	20	25	30	35	40	45
6	0	6	12	18	24	30	36	42	48	54
7	0	7	14	21	28	35	42	49	56	63
8	0	8	16	24	32	40	48	56	64	72
9	0	9	18	27	36	45	54	63	72	81

Figure 5-5: A lookup table, such as this table of products of all single-digit numbers, saves our program from repeat calculations as the computer stores the precomputed values in memory for later retrieval.

We'll implement Karatsuba multiplication in a high-level language like Python or JavaScript as though the `*` operator didn't already exist. Our `karatsuba()` function accepts two integer arguments, `x` and `y`, to multiply. The Karatsuba algorithm has five steps, and the first three involve making recursive calls to `karatsuba()` with arguments that are smaller, broken-down integers derived from `x` and `y`. The base case occurs when the `x` and `y` arguments are both single-digit numbers, in which case the product can be found in the precomputed lookup table.

We also define four more variables: `a` and `b` are each half of the digits of `x`, and `c` and `d` are each half of the digits of `y`, as shown in [Figure 5-6](#). For example, if `x` and `y` are 5678 and 1234, respectively, then `a` is 56, `b` is 78, `c` is 12, and `d` is 34.

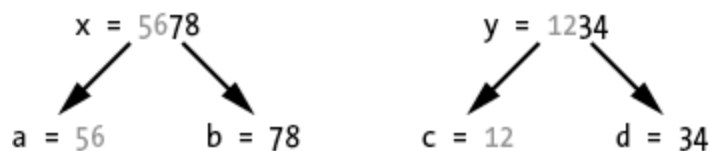


Figure 5-6: The integers to multiply, `x` and `y`, are divided into halves `a`, `b`, `c`, and `d`.

Here are the five steps of the Karatsuba algorithm: